K8S Revisited

Object	Purpose
Deployment	Manages replicas, rolling updates, and scaling of pods.
Service	Exposes a set of pods as a stable network endpoint.
ConfigMap	Stores non-sensitive configuration data separately from app code.
NetworkPolicy	Defines rules for allowed/denied network traffic between pods.

Q1. Can I deploy Python, React, and Go app pods in a cluster with --num-nodes=1?

✅ **Yes, you can!**

- **Nodes** are the actual machines (VMs in GKE) that run your applications.
- **Pods** are the smallest deployable units in Kubernetes that contain your app containers.
- A single node can run **multiple pods** as long as there are enough **CPU and memory resources**.

💡 **Key Point:** If your Python, React, and Go apps have small resource requirements, they **can all run on a single node**. But if they demand more CPU/memory than available, some pods may be **Pending** (waiting for resources).

📌 **Example Scenario:**

App	Pod Count	CPU/Memory Requests	Fits in 1 Node?
Python API	1 Pod	0.5 CPU / 256 MB	✅ Yes
React Frontend	1 Pod	0.3 CPU / 512 MB	✅ Yes
Go Service	1 Pod	0.7 CPU / 512 MB	✅ Yes

👉 **All can run in 1 node, but if resource usage increases, the node may become full.**

Q2. If replicaSet=2 for each app, do pods scale up within a single node or across multiple nodes?

It depends:

✅ **If there is enough CPU & RAM on the single node**, both replicas will be scheduled on the same node.

❌ **If the node runs out of resources**, new pods will remain **Pending** until more nodes are added (via autoscaling or manual scaling).

✅ **If the cluster has multiple nodes**, the pods **distribute** across available nodes to balance load.

📌 **Example Scenario (--num-nodes=1, replicaSet=2 for each app)**

App	Pod Count	CPU/Memory Requests per Pod	Node Fits Both Pods?
Python API	2 Pods	0.5 CPU / 256 MB	✅ Yes
React Frontend	2 Pods	0.3 CPU / 512 MB	✅ Yes
Go Service	2 Pods	0.7 CPU / 512 MB	❌ No (will be Pending)

👉 If the node **cannot fit all replicas**, some pods **won't run** unless more nodes are added.

🔷 **Solution:**

- Enable **Cluster Autoscaler** (--enable-autoscaling --min-nodes=1 --max-nodes=5).
- Use kubectl describe node to check resource usage.

Q3. How to check which pods are running on which node?

To list all pods and see which node they are assigned to:

kubectl get pods -o wide

📌 Example Output:

sql
CopyEdit

```
NAME                    READY   STATUS    NODE
python-api-7fdc4c9c6b-1  1/1     Running   gke-node-1
python-api-7fdc4c9c6b-2  1/1     Running   gke-node-1
react-frontend-8bc9c7c6b-1  1/1 Running   gke-node-2
go-service-9ac3c8b7d-1   1/1     Pending   <none>
```

👉 This tells us which **node** each pod is running on.

If you want to list all nodes and their allocated pods:

kubectl get nodes
kubectl describe node <node-name>

Q5. Are the Control Plane and Data Plane nodes or pods?

✅ **Control Plane = Nodes (managed by GKE, NOT visible to you)**

✅ **Data Plane = Nodes (Worker Nodes where your Pods run)**

- **Control Plane Nodes** (Managed by GKE)
  - These are special nodes that manage your cluster.
  - You don't interact with them directly in **GKE** (Google handles them).
  - Runs components like API Server, Scheduler, Controller Manager, etc.
- **Data Plane Nodes** (Worker Nodes)
  - These are actual **VMs (Nodes)** that run your application Pods.
  - You can see them using kubectl get nodes.

💡 **Summary:**

Plane	What It Contains	Managed By
Control Plane	API Server, etcd, Scheduler, Controllers	Google (GKE)
Data Plane	Worker Nodes (running your app pods)	You (User)

Q6: What is a Pod in Kubernetes?

A **Pod** is the **smallest deployable unit** in Kubernetes. It is an **abstraction** over one or more containers that share the **same network namespace, storage, and lifecycle**.

🔷 Pod vs Node vs Container

Component	What it Represents?
Node	A physical/virtual machine (CPU, RAM, Disk)
Pod	A group of one or more **containers** running together
Container	A lightweight, isolated process running an application (e.g., Docker container)

📌 **Key Points about Pods:**

- **Runs multiple containers** (e.g., a main app + a sidecar for logging).
- **Shares storage volumes** across all containers.
- **Shares networking** (all containers inside the Pod communicate via localhost).
- **Manages resource limits** via Kubernetes settings.

Q7. How Does a Pod Track CPU & RAM Usage?

Even though CPU and RAM are allocated at the **Node level**, Kubernetes tracks **Pod-level resource consumption** using **cgroups (Control Groups)**.

1️⃣ Container Runtime (e.g., Docker, containerd) Uses cgroups

- Every **Pod runs in its own cgroup**, which defines resource limits for all its containers.
- The container runtime (e.g., containerd, Docker) uses **Linux cgroups** to track CPU, memory, and disk usage.

📌 **Example: View Pod Resource Usage using cgroups**

```
cat /sys/fs/cgroup/cpu/kubepods/burstable/pod<UID>/cpuacct.usage
cat /sys/fs/cgroup/memory/kubepods/burstable/pod<UID>/memory.usage_in_bytes
```

➡️ This shows the **CPU and memory used by the Pod**.

Q8. How Does a Pod Autoscale If CPU is Tracked at Node Level?

Kubernetes provides **Horizontal Pod Autoscaling (HPA)** to scale Pods **based on CPU and memory usage**.

2️⃣ Kubernetes Monitors Pod Resource Usage

- Kubernetes **keeps track of CPU and memory usage** through the **Metrics Server**.
- The **Metrics Server** collects data from kubelet on each Node.
- **HPA (Horizontal Pod Autoscaler)** scales Pods when CPU/memory crosses a threshold.

📌 **Example: Enable Autoscaling for a Pod**

```
kubectl autoscale deployment my-app --cpu-percent=50 --min=2 --max=5
```

➡️ If CPU **exceeds 50%**, Kubernetes adds more **Pods**.

Q9. What is the Metrics Server? Where is it Located?

The **Metrics Server** is a Kubernetes component that **collects and aggregates resource usage data** (CPU, memory) from nodes and pods. It is used by:

✅ **Horizontal Pod Autoscaler (HPA)** → to scale pods based on CPU/memory.

✅ **Vertical Pod Autoscaler (VPA)** → to adjust pod resource requests/limits.
✅ **kubectl top** → to check pod and node resource usage.

Q10. Where is the Metrics Server Located?

The Metrics Server **runs as a Deployment inside the cluster**.
📌 It does **not store data** persistently (unlike Prometheus).
To check if it's running:

kubectl get deployment -n kube-system | grep metrics-server

If not installed, you can deploy it:

kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

Q11. How Does the Metrics Server Work?

1. **Each Node has a kubelet process** that exposes pod/node resource usage via the /metrics/ resource API.
2. **Metrics Server scrapes this data** periodically.
3. **It aggregates and exposes the data via the Kubernetes API** at:

   https://<metrics-server-ip>/apis/metrics.k8s.io/v1beta1/
4. **HPA or kubectl top** fetches data from this API.

🔷 Example Commands

Check Node Resource Usage

kubectl top nodes

Check Pod Resource Usage

kubectl top pods --all-namespaces

Check API Response (Raw Metrics)

kubectl get --raw "/apis/metrics.k8s.io/v1beta1/node

Q12: What is cgroup at Container and Pod Level?

cgroup (Control Group) is a **Linux kernel feature** that **limits, accounts, and isolates** CPU, memory, disk, and network usage for a group of processes. It is used by **Docker, containerd, and Kubernetes** to enforce resource limits at both **container** and **Pod** levels.

cgroup at the Container Level

Each **container** in Kubernetes runs inside a **Linux cgroup**, which is managed by the container runtime (e.g., containerd, Docker).

- cgroups ensure a **container does not exceed its allocated resources** (CPU, memory, etc.).
- Every container has its own **cgroup directory** inside /sys/fs/cgroup/.

📌 **Example: View CPU & Memory Usage of a Running Container**

docker stats

This command shows real-time **CPU, memory, and network usage** of containers.

📌 **Manually Check cgroups for a Container**

cat /sys/fs/cgroup/cpu/docker/<container-id>/cpuacct.usage
cat /sys/fs/cgroup/memory/docker/<container-id>/memory.usage_in_bytes

➡️ This gives the **CPU time (in nanoseconds) and memory usage (in bytes).**

cgroup at the Pod Level

Since **a Pod can run multiple containers**, Kubernetes creates **a parent cgroup** for the entire **Pod** and assigns child cgroups to individual **containers** inside it.

How Kubernetes Organizes cgroups for Pods and Containers

1. **Pod cgroup** → A directory for the entire Pod.
2. **Container cgroup** → A subdirectory inside the Pod cgroup.

📌 **Kubernetes cgroup structure example (kubepods):**

/sys/fs/cgroup/cpu/kubepods/burstable/pod<UID>/container<Container-ID>

- **kubepods/** → Contains all Pods in Kubernetes.
- **burstable/** → QoS class (Guaranteed, Burstable, or BestEffort).
- **pod<UID>/** → Pod-specific cgroup (tracks resource usage of the Pod).
- **container<Container-ID>/** → Container-specific cgroup (tracks container resources inside the Pod).

📌 **Example: View CPU & Memory Usage of a Pod**

cat /sys/fs/cgroup/cpu/kubepods/burstable/pod<UID>/cpuacct.usage
cat /sys/fs/cgroup/memory/kubepods/burstable/pod<UID>/memory.usage_in_bytes

➡️ This gives **aggregated CPU and memory usage of all containers inside the Pod.**

Q13: How Does Network and Storage Work Between Sidecar and Main Containers?

Great question! Let's break it down.

🔷 **Does a Sidecar Container Share Storage with the Main Container?**

✅ **Yes, but only if a shared volume is explicitly mounted.**

✅ Containers within the same Pod **do not have isolated filesystems** by default.

✅ They **can share storage via volumes** (emptyDir, PersistentVolume, etc.).

✅ **Containers in different Pods (like c1 and c3) do NOT share storage**.

🔷 **Understanding Your Node Structure**

Let's assume your **Node (n1)** has **two Pods (p1, p2)** and multiple containers:

```
n1
 ├──── p1
 |     ├──── c1 (Main App)
 |     ├──── c2 (Sidecar)
 |
 ├──── p2
 |     ├──── c3
 |     ├──── c4
 |     ├──── c5
```

🔷 **How Storage Works in This Setup**

Scenario	Storage Sharing?	Why?
c1 ↔ c2 (Same Pod)	✅ Yes	Uses a shared volume (emptyDir, PVC, etc.)
c1 ↔ c3 (Different Pods)	❌ No	Pods are isolated from each other
c3 ↔ c4 (Same Pod)	✅ Yes	If they mount the same volume
c1 ↔ Node Storage	❌ No	Pods cannot directly access Node's filesystem (except HostPath)

What About localhost Communication?

✅ **Containers inside the same Pod can communicate over localhost**.

✅ Example:

- c1 exposes a service on localhost:5000.
- c2 can reach it via http://localhost:5000.

❌ **Containers in different Pods cannot communicate via localhost**.

✅ Instead, they need a **Service** (ClusterIP) to communicate.

Q13: What is a Sidecar in Kubernetes?

A **Sidecar** is a container that runs **alongside the main application container** in the same **Pod** to provide **supporting functionalities** such as logging, monitoring, security, or networking.

Where is a Sidecar Deployed?

- **Level: Pod** (not Node or Cluster level).
- **Runs inside the same Pod** as the primary container.
- **Example:** A **logging agent** that collects logs from the application container.

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app-pod
spec:
  containers:
    - name: main-app
      image: my-app:latest
      ports:
        - containerPort: 8080
    - name: sidecar-logger
      image: busybox
      command: ["/bin/sh", "-c"]
      args: ["while true; do cat /var/log/app.log; sleep 10; done"]
      volumeMounts:
        - name: log-volume
          mountPath: /var/log
  volumes:
    - name: log-volume
      emptyDir: {}
```

Q14. If I enable --enable-autoscaling --min-nodes=1 --max-nodes=5, does it mean node scaling?

✅ **Yes, this enables Node Scaling (Cluster Autoscaler).**

What happens when a new node is added?

- Pods **are NOT copied** from one node to another.

- Instead, **only new pods** are scheduled on new nodes.

Who manages this movement?

- **Kubernetes Scheduler**
  - It decides where **new pods should go** based on resource availability.
  - It doesn't move existing pods unless a node **fails**.

💡 **Example:**

Nodes	Pods Running	Autoscaler Decision
1 Node	5 Pods	✅ Fine
1 Node	10 Pods (overloaded)	➡️ Adds a 2nd node
2 Nodes	12 Pods (more load)	➡️ Adds a 3rd node
3 Nodes	Load reduces to 6 Pods	❌ Removes extra nodes

🔷 **Pods stay in their original nodes unless manually rescheduled.**

🔷 **When nodes are removed, their pods are terminated and rescheduled.**

Q15. How does traffic split when multiple replicas exist across nodes?

Let's break this down step by step:

Scenario:

- **Replica 1 (python-api-1) → Node 1**
- **Replica 2 (python-api-2) → Node 2**
- **HPA is enabled** (pods scale up/down).
- **Load Balancer is set up**.

Who distributes traffic?

✅ **Kubernetes Service (LoadBalancer or ClusterIP)**

✅ **It is a separate service, NOT a pod**

Where does this Load Balancer exist?

- If you use **Google Cloud Load Balancer**, it exists **outside** the cluster.
- If you use **Kubernetes Service (LoadBalancer or ClusterIP)**, it exists **inside the cluster**.

How does traffic split?

1️⃣ **External Traffic (Internet Users)**

- Users hit **Google Cloud Load Balancer** (external).
- Load Balancer sends traffic to a **Kubernetes Service** (type=LoadBalancer).
- The Service forwards requests **evenly** to available pods.

```
apiVersion: v1
kind: Service
metadata:
  name: python-api-service
spec:
  type: LoadBalancer
  selector:
    app: python-api
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
```

2️⃣ **Internal Traffic (Within Cluster, Between Pods)**

- A ClusterIP service directs traffic to pods.
- Kubernetes uses **round-robin load balancing** by default.

💡 **Key Notes:**

- **LoadBalancer Service = Handles traffic from users.**
- **ClusterIP Service = Handles traffic between services inside the cluster.**
- **Pods in different nodes receive traffic evenly.**

📌 **Example:**

Request	Load Balancer Sends To
Request 1	python-api-1 (Node 1)
Request 2	python-api-2 (Node 2)
Request 3	python-api-1 (Node 1)
Request 4	python-api-2 (Node 2)

Q16. How does KubeDNS (or CoreDNS) work? What is its role?

✅ **KubeDNS (or CoreDNS) is Kubernetes' internal DNS system.**

✅ **It resolves service names to their internal IPs.**

Why do we need KubeDNS/CoreDNS?

- In a Kubernetes cluster, **Pods have dynamic IPs** (they change if a Pod restarts).

- Instead of hardcoding Pod IPs, **Services** provide a **stable DNS name**.
- **KubeDNS/CoreDNS resolves these service names** into actual Pod IPs dynamically.

How does it work?

1️⃣ **Every Service gets a DNS entry**

- Example: If you create a service called python-api, it gets a DNS name likepgsql
  CopyEdit

  python-api.default.svc.cluster.local

- If another Pod wants to access it, it just usessh
  CopyEdit

  curl http://python-api

  instead of worrying about IP addresses.

Q17. What is GKE? How does it manage clusters? What happens when I run gcloud container clusters create?

✅ **GKE (Google Kubernetes Engine) is a managed Kubernetes service by Google Cloud.**

✅ It **automates** cluster setup, node provisioning, scaling, and maintenance.

✅ When you create a cluster with gcloud container clusters create, GKE sets up:

- The **Control Plane** (API Server, etcd, Scheduler, etc.)
- The **Worker Nodes** (where your pods will run)

What happens when you run this command?

```
gcloud container clusters create $GKE_CLUSTER_NAME \
  --region $GAR_LOCATION \
  --num-nodes 3 \
  --enable-ip-alias \
  --machine-type=e2-standard-2
```

🔷 **Step 1: GKE Creates the Control Plane**

- A **Control Plane (Master Node)** is created (you don't see this, GKE manages it).
- It includes:
  - API Server (handles commands)
  - Scheduler (assigns Pods to Nodes)
  - Controller Manager (manages cluster state)
  - etcd (stores cluster state data)

🔷 **Step 2: GKE Creates 3 Worker Nodes**

- These are **Compute Engine (VMs) instances** that act as worker nodes.
- Each node runs:
  - kubelet (node agent that talks to Control Plane)

- kube-proxy (handles networking & service discovery)
- Container Runtime (like Docker or containerd to run containers)

🔷 **Step 3: Networking is Configured (--enable-ip-alias)**

- Enables VPC-native networking.
- Each **Pod gets a unique IP** from a **VPC subnet** instead of being NAT'd.

🔷 **Step 4: The Cluster is Ready!** 🎉

- The Control Plane is ready to schedule Pods.
- Nodes are waiting for workloads.
- You can now deploy applications!

Who allocates all these resources?

✅ **Google Kubernetes Engine (GKE) automatically provisions the Control Plane & Worker Nodes.**

✅ It integrates with **Google Compute Engine** to allocate VMs for Worker Nodes.

✅ It sets up a **private VPC subnet** for Kubernetes networking.

Q18. What is a Fleet and Namespace in Kubernetes?

Fleet in Kubernetes (GKE Fleet)

✅ **Fleet is a way to manage multiple GKE clusters together** as a single logical entity.

✅ It's useful for **multi-cluster management**, security, and networking.

✅ It enables **Anthos Service Mesh, Config Sync, and IAM policies across clusters**.

📌 **Key Features of a Fleet**

- **Common Policies:** Apply IAM, RBAC, and security policies to all clusters in a Fleet.
- **Multi-cluster Networking:** Connect clusters across regions without complex setup.
- **Anthos Service Mesh:** Unified service discovery and traffic management across clusters.
- **Workload Identity Federation:** Secure communication between GCP services and Kubernetes.

Namespace in Kubernetes

✅ **A Namespace is a logical grouping of Kubernetes resources.**

✅ It is used to **organize** workloads, apply different policies, and manage RBAC.

✅ By default, everything is deployed into the **default namespace** unless specified.

📌 **Use Cases of Namespaces**

- **Multi-Tenant Clusters:** Different teams (e.g., dev, staging, prod) in the same cluster.
- **Resource Quotas:** Limit CPU, memory, or storage usage per namespace.
- **Isolation:** Prevent one workload from affecting another.

🔷 **Example: Creating a Namespace**

kubectl create namespace my-app

🔷 **Deploying into a Namespace**

kubectl apply -f my-deployment.yml --namespace=my-app

🔷 **Viewing Resources in a Namespace**

kubectl get pods -n my-app

Q19. What happens when I specify no-proxy in values.yml? Who manages this proxy? What is no-proxy in values.yml?

✅ no-proxy is used to **bypass proxy settings** for certain destinations.

✅ It tells the application or Kubernetes **not** to route traffic through a proxy.

📌 **Where is no-proxy used?**

- **Helm charts (values.yml)**: Used to configure proxies for applications.
- **Kubernetes Environment Variables**:
  - HTTP_PROXY
  - HTTPS_PROXY
  - NO_PROXY

🔷 **Example values.yml with no-proxy**

```
global:
  proxy:
    httpProxy: "http://proxy.example.com:3128"
    httpsProxy: "https://proxy.example.com:3128"
    noProxy: "10.0.0.0/8,localhost,127.0.0.1"
```

📌 **How it is resolved?**

1️⃣ The Kubernetes Pod uses the **proxy settings** from environment variables.

2️⃣ If a request **matches the no-proxy list**, it **bypasses the proxy**.

3️⃣ Otherwise, it **routes traffic through the proxy**.

📌 **Who manages the proxy?**

- **Network administrators** usually configure proxies.
- **Istio or Envoy proxies** can be used inside Kubernetes clusters.
- **GKE Autopilot** may set up network proxies automatically for security.

Q20. Who maintains the routing table? Is there a router inside Kubernetes?

Who maintains the routing table in Kubernetes?

✅ Kubernetes uses **kube-proxy and iptables/ipvs to manage routing tables.**

✅ The **Linux kernel** handles packet forwarding based on these rules.

✅ In cloud environments (like GKE), **Google Cloud VPC routing** also plays a role.

Is there a router inside Kubernetes?

✅ **Yes, but it's not a traditional router.**

✅ Kubernetes uses **kube-proxy** as a **networking layer** for routing inside the cluster.

📌 **How does routing work in Kubernetes?**

1️⃣ **Pod-to-Pod Communication**

- The **CNI (Container Network Interface)** plugin (like Calico, Cilium, Flannel) manages routes.
- Every Pod gets a **unique IP** inside the cluster.
- Traffic is **routed directly** to the destination Pod via the CNI network.

2️⃣ **Service Routing (kube-proxy)**

- kube-proxy uses **iptables/ipvs** rules to route traffic to Services.
- A **Service (ClusterIP, NodePort, LoadBalancer)** determines how traffic is forwarded.

3️⃣ **External Traffic Routing**

- External requests go through a **LoadBalancer (GKE Load Balancer or Ingress Controller)**.
- The **Cloud VPC routing** handles external traffic into the cluster.

🔷 **Checking Kubernetes Routing Tables**

```
kubectl get svc -A
kubectl describe svc my-service
```

🔷 **Checking Linux Routing Table inside a Pod**

```
kubectl exec -it my-pod -- ip route
```

Q21: How does kube-proxy handle traffic and deal with ClusterIP, External IP, NodePort, and ContainerPort?

What is kube-proxy?

✅ kube-proxy is a **networking component in Kubernetes** that routes traffic between **Pods and Services**.

✅ It manages networking using **iptables (default), ipvs, or userspace mode**.

✅ It ensures that traffic is forwarded correctly based on **Kubernetes Service types (ClusterIP, NodePort, LoadBalancer, External IPs, etc.)**.

How kube-proxy Handles Routing

1️⃣ **kube-proxy with iptables (Default)**

✅ Uses **NAT (Network Address Translation)** to route traffic.

✅ Rules are stored in **iptables** for fast packet forwarding.

✅ Less overhead, but **does not handle millions of connections well**.

🔷 **Check iptables Rules for a Service:**

```
iptables -t nat -L -n -v
```

2️⃣ **kube-proxy with IPVS**

✅ Uses **IPVS (IP Virtual Server)**, a high-performance Linux kernel feature.

✅ Handles **millions of connections** more efficiently than iptables.

✅ Requires --proxy-mode=ipvs enabled in kube-proxy.

🔷 **Check IPVS Routing Table:**

```
ipvsadm -L -n
```

Concept	Description
ClusterIP	Internal communication only (kube-proxy routes traffic using ClusterIP).
External IP	Exposes a Service outside the cluster (kube-proxy routes requests from an external IP).
NodePort	Opens a high port (30000-32767) on each node to expose a Service with Static IPs
ContainerPort	Defines the port inside the Pod that receives traffic (not exposed outside the cluster).

kube-proxy	Manages networking using **iptables/ipvs** to forward traffic between Pods and Services.

Want to debug your networking?

🔷 Check which Services exist:

kubectl get svc -A

🔷 Describe a Service:

kubectl describe svc my-service

🔷 Find which Pods are running on which Nodes:

kubectl get pods -o wide

Q22: Is ClusterIP for a Cluster or a Pod? How are Pod addresses resolved? How does the internal DNS (/etc/hosts) look?

📌 Is ClusterIP for a Cluster or a Pod?

✅ **ClusterIP is assigned to a Kubernetes Service, not a Pod.**

✅ It is a **virtual IP** that allows communication **within the cluster**.

✅ **Pods do NOT get ClusterIPs.** They have their **own individual IPs** assigned by the network.

📌 **Example:**

- **Service (my-service) has a ClusterIP**: 10.96.0.10
- **Pods behind this Service have individual IPs**: 10.244.1.10, 10.244.2.12, etc.
- Any Pod inside the cluster can call http://my-service:port, and kube-proxy will route the request to one of the **backend Pods**.

📌 How are Pod Addresses Resolved?

Kubernetes **does not use ClusterIP to resolve Pod addresses directly**. Instead, it relies on **internal DNS (kube-dns or CoreDNS)** to map names to IPs.

1️⃣ Pod-to-Pod Communication

✅ Each Pod gets a **unique IP** (assigned by CNI like Calico, Flannel, Cilium).

✅ Pods **do NOT need ClusterIP** to communicate directly; they can use the **Pod IP**.

✅ Pods communicate **across nodes** using **overlay networking (VXLAN, etc.)**.

🔷 **Find Pod IPs**:

kubectl get pods -o wide

🔷 **Example:**

NAME	READY	STATUS	IP	NODE
pod-1	1/1	Running	10.244.1.10	node-1
pod-2	1/1	Running	10.244.2.12	node-2

- Pod-1 can directly call 10.244.2.12:8080 if the network allows it.

2️⃣ Pod-to-Service Communication (via ClusterIP)

✅ **Pods never need to know the individual IPs of other Pods.**

✅ Instead, they talk to **Services**, which act as a stable entry point.

✅ **kube-dns/CoreDNS resolves the Service name to the ClusterIP**.

✅ **kube-proxy** forwards traffic to a healthy Pod.

🔷 **Example:**

A Pod wants to talk to **my-service**:

curl http://my-service:8080

- kube-dns resolves my-service → 10.96.0.10 (ClusterIP).
- kube-proxy forwards the request to **one of the backend Pods**.

📌 What does the internal /etc/hosts file look like?

Each **Pod** has its own **/etc/hosts** file, which contains basic **hostname-IP mappings**.

Example /etc/hosts inside a Pod:

127.0.0.1	localhost
10.244.1.10	pod-1
10.244.2.12	pod-2

📌 **But Kubernetes doesn't rely on /etc/hosts for service discovery**. It uses **DNS (CoreDNS or kube-dns)**.

📌 How does the Internal DNS Table look like?

The **internal DNS (kube-dns or CoreDNS)** provides domain name resolution inside Kubernetes.

1️⃣ Service DNS Format

pgsql
CopyEdit
<Service-Name>.<Namespace>.svc.cluster.local

🔷 Example DNS resolution for a Service (**my-service** in namespace **default**):

nslookup my-service.default.svc.cluster.local
Returns:
Name: my-service.default.svc.cluster.local
Address: 10.96.0.10 # (ClusterIP of Service)

2️⃣ Pod DNS Format

<Pod-IP>.<Namespace>.pod.cluster.local

🔷 Example:

nslookup 10-244-1-10.default.pod.cluster.local
Returns:

Name: 10-244-1-10.default.pod.cluster.local
Address: 10.244.1.10

📌 Final Summary

Concept	Description
ClusterIP	A stable **virtual IP** for a Service, used **inside** the cluster.
Pod IPs	Assigned dynamically by the CNI. Pods use their own IPs for direct communication.
Pod Discovery	Pods are discovered via **CoreDNS (kube-dns)**, not /etc/hosts.
DNS Resolution	Services follow <service>.<namespace>.svc.cluster.local, while Pods follow <pod-ip>.<namespace>.pod.cluster.local.

🚀 **Want to check DNS in your cluster?**

kubectl exec -it <pod-name> -- nslookup my-service.default.svc.cluster.local

Q23: What is a Service in Kubernetes? Is it a Pod, Node, or Cluster?

📌 **A Service in Kubernetes is NOT a Pod, NOT a Node, and NOT a Cluster.**

✅ It is an **abstraction** that provides **a stable network endpoint** to access a set of Pods.

📌 **What is the Purpose of a Kubernetes Service?**

- Pods are **ephemeral** (they can be created/destroyed dynamically).
- Each Pod gets a **unique IP**, but **Pod IPs are not stable** (they change when a Pod restarts).
- **A Service ensures a stable network identity** for a group of Pods.
- It **load balances traffic** between multiple Pods.

What does a Kubernetes Service do?

✅ **Provides a stable IP & DNS name** (ClusterIP)

✅ **Load balances traffic** across multiple Pods

✅ **Enables communication between Pods** within or outside the cluster

📌 Types of Kubernetes Services

Service Type	Description	Use Case
ClusterIP (default)	Internal communication between Pods in the same cluster	Backend APIs, Databases
NodePort	Exposes Service on a static port on each Node	Direct access to Service from outside
LoadBalancer	Uses a cloud provider's Load Balancer	Exposes app to the internet
ExternalName	Maps Service to an external DNS name	Proxying external services

📌 Example: Defining a Service

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: python-app  # Selects Pods with this label
  ports:
    - protocol: TCP
      port: 80       # Service port (ClusterIP)
```

```
  targetPort: 5000  # Port inside Pod
```

- **selector: app=python-app** → Finds matching Pods
- **port: 80** → Clients use this port to access the Service
- **targetPort: 5000** → Forwards traffic to Pods' **port 5000**

Q24. Questions:

Can two containers within the same pod have exclusive storage volumes? If so, when one container makes a request to localhost/getdata, will it always retrieve data from its own volume, or is there a risk of accessing the other container's data due to shared networking? How does the behavior of accessing localhost/getdata differ between multiple replicas of a pod and multiple containers inside a single pod?

Difference in localhost/getdata Behavior for Pods vs. Containers

Aspect	Multiple Pods (Replicas of a Deployment)	Multiple Containers in the Same Pod
Network Namespace	Each pod has its own network namespace.	Containers share the same network namespace.
Storage	Each pod can have its own storage volume.	Containers can have separate or shared storage within the pod.
Accessing localhost/getdata	Request stays within the pod; handled by the container listening on the port.	Request is received by whichever container is bound to the requested port.
Port Binding	Each pod binds ports separately (e.g., 8000for each replica).	All containers inside a pod share the same localhost, so they must bind to different ports.
Traffic Routing	A **Service** (ClusterIP, LoadBalancer) handles traffic distribution between pods.	No service needed; direct inter-container communication via localhost:<port>.
Risk of Data Inconsistency	No risk unless storage is shared between pods.	Possible if containers share the same volume, as they will see each other's files.

🚀 **Key Takeaways:**

- **Pods do not share localhost; containers inside a pod do.**
- **For inter-container communication in the same pod, use different ports (e.g., 8000, 8001).**
- **Pods should use services (ClusterIP, LoadBalancer) to expose and distribute traffic.**
- **Data inconsistency happens when multiple containers in a pod share storage.**

```
volumes:
- name: shared-storage
  emptyDir: {}

containers:
- name: c1
  volumeMounts:
  - mountPath: "/data"
    name: shared-storage

- name: c2
  volumeMounts:
  - mountPath: "/data"
    name: shared-storage
```

🚨 **If they both use /data as a shared volume, c1 could read c2's data and vice versa.** 🚨
If c1 writes to /data/file.txt, c2 can read it immediately.

✅ To prevent conflicts, **each container should bind to a unique port**:

```
containers:
```

```
- name: c1
  image: my-app
  ports:
  - containerPort: 8000

- name: c2
  image: my-app
  ports:
  - containerPort: 8001
```

Q25: How does traffic flow from the outside world into a Kubernetes Cluster?

To fully understand this, let's break it down into **all layers, IPs, and forwarding mechanisms** from **External Traffic → LoadBalancer → Cluster → kube-proxy → Node → Pod → Container**.

🔷 Step-by-Step Traffic Flow in Kubernetes

1️⃣ External Traffic Enters the Cloud Provider's Load Balancer

- **Component:** Cloud Provider's **External LoadBalancer** (Managed by GKE, AWS, Azure, etc.)
- **IP Address: External IP** (Assigned by the cloud provider)
- **Ports:** Typically **80 (HTTP) or 443 (HTTPS)**
- **Purpose:** Exposes the Kubernetes Service to the internet and forwards requests inside the cluster.

📌 **Example:**

kubectl get svc

NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
my-service LoadBalancer 10.100.0.1 34.120.150.45 80:31234/TCP

➡️ Here, **34.120.150.45** is the **public IP** of the LoadBalancer.

2️⃣ LoadBalancer forwards traffic to a Node via NodePort

- **Component:** Kubernetes Service of type **LoadBalancer**
- **IP Address: Node External IP** (Public IP of the VM/Node)
- **Ports: NodePort** (e.g., 31234)
- **Purpose:** The LoadBalancer forwards traffic to a **random Node** that runs the Service.

📌 **Example:**

```
spec:
  type: LoadBalancer
  ports:
    - port: 80       # External port
      targetPort: 5000 # Inside Pod
      nodePort: 31234  # Exposed on each Node
```

➡️ The LoadBalancer picks **one of the Nodes** and forwards traffic to **NodePort 31234**.

3️⃣ Node receives the traffic and forwards it to kube-proxy

- **Component:** Worker Node running Kubernetes
- **IP Address: Node Internal IP** (e.g., 192.168.1.10)
- **Ports: NodePort (31234)**
- **Purpose:** The Node receives traffic on its **NodePort** and forwards it to the kube-proxy.

📌 **Example:**

```
Node 1 (192.168.1.10) -- receives traffic on 31234
```

➡️ Traffic arrives at **192.168.1.10:31234**, and **kube-proxy** takes over.

4️⃣ kube-proxy forwards traffic to the ClusterIP of the Service

- **Component:** kube-proxy (handles internal routing in the cluster)
- **IP Address: ClusterIP of the Service** (e.g., 10.100.0.1)
- **Ports: Service Port (80)**
- **Purpose:** kube-proxy forwards traffic from the NodePort (31234) to the **ClusterIP** (10.100.0.1).

📌 **Example:**

kube-proxy NATs traffic:

192.168.1.10:31234 → 10.100.0.1:80

➡️ The Service forwards the request **to one of the Pods**.

5️⃣ Service forwards traffic to a Pod IP

- **Component:** Kubernetes Service
- **IP Address: Pod IP** (e.g., 10.244.2.15)
- **Ports: Pod TargetPort (5000)**
- **Purpose:** The Service selects **one of the available Pods** using **round-robin or another strategy**.

📌 **Example:**

Load Balancing:

10.100.0.1:80 → 10.244.2.15:5000

➡️ Now, traffic **reaches the correct Pod**.

6️⃣ Pod forwards traffic to the Container Port

- **Component:** The Pod running inside the Node
- **IP Address: Container's localhost (127.0.0.1)**
- **Ports: Container Port (5000)**
- **Purpose:** The Pod finally forwards traffic to **the running container's process**.

📌 **Example:**

Pod 10.244.2.15:5000 → Container 127.0.0.1:5000

➡️ The container **processes the request**.

📌 Summary of Layers, IPs, and Port Mappings

Layer	Component	IP Address	Port	Purpose
1️⃣ **LoadBalancer**	Cloud LoadBalancer (GCP, AWS, Azure)	External IP (e.g., 34.120.150.45)	80 → 31234	Exposes Kubernetes Service
2️⃣ **Node**	Kubernetes Worker Node	Node External IP (192.168.1.10)	31234	Accepts traffic on NodePort
3️⃣ **kube-proxy**	Internal Kubernetes Proxy	ClusterIP (10.100.0.1)	80	Routes traffic inside cluster

4️⃣ **Service**	Kubernetes Service	Pod IP (10.244.2.15)	5000	Load balances across Pods
5️⃣ **Pod**	Kubernetes Pod	Container 127.0.0.1	5000	Runs the application

Q26. Traffic Distribution Responsibilities in Kubernetes ?

Component	Scope of Distribution	How It Works
LoadBalancer	Distributes traffic among **nodes**	External traffic enters via a cloud provider's **LoadBalancer**, which forwards it to a **NodePort**.
NodePort	Exposes a service on all nodes	A **NodePort service** makes a service accessible on NodeIP:NodePort, forwarding to a pod.
ClusterIP	Distributes traffic among **pods**	Default service type, allowing internal communication between pods.
kube-proxy	Handles service routing	Sets up **iptables/ipvs** rules to ensure traffic reaches the correct pod.

What If LoadBalancer Routes to a Node Without the Service?

- LoadBalancers forward traffic to a **NodePort**, which is exposed on **all nodes**.
- Even if the actual pod is not on that node, **kube-proxy** ensures the traffic is forwarded to the correct pod on another node.

🚀 **Example Flow:**

1. External request → LoadBalancer (chooses a node).
2. LoadBalancer forwards to NodeIP:NodePort.
3. **If pod exists on the node**, request goes to it directly.
4. **If pod does not exist**, kube-proxy reroutes traffic to the correct node where the pod is running.

Q27. How kube-proxy Handles Traffic with iptables and ipvs 🚀

kube-proxy is responsible for **service-to-pod traffic routing** inside a Kubernetes cluster. It ensures that requests sent to a **service** reach the correct pod(s) by managing **iptables** (or **IPVS**) rules on each node.

1. Using iptables Mode

By default, kube-proxy uses **iptables** to route traffic inside the cluster.

Example Scenario

You have a **ClusterIP service** named my-service that maps to two pods:

- Pod-A at 10.244.1.2
- Pod-B at 10.244.2.3
- **ClusterIP:** 10.96.0.10
- **Port:** 80

💡 When a pod tries to access 10.96.0.10:80, **iptables rules** redirect the request to **one of the backend pods**.

How iptables Handles It

```
iptables -t nat -A KUBE-SERVICES -d 10.96.0.10/32 -p tcp --dport 80 -j KUBE-SVC-XYZ
iptables -t nat -A KUBE-SVC-XYZ -m statistic --mode random --probability 0.5 -j KUBE-SEP-AAA
iptables -t nat -A KUBE-SVC-XYZ -j KUBE-SEP-BBB
iptables -t nat -A KUBE-SEP-AAA -p tcp -j DNAT --to-destination 10.244.1.2:80
iptables -t nat -A KUBE-SEP-BBB -p tcp -j DNAT --to-destination 10.244.2.3:80
```

Explanation

1. Traffic sent to 10.96.0.10:80 is caught by **KUBE-SERVICES**.
2. **KUBE-SVC-XYZ** selects a pod based on a **random probability rule**.
3. If selected, it **DNATs** (Destination NAT) the request to the correct pod (Pod-A or Pod-B).

2. Using ipvs Mode

IPVS (IP Virtual Server) is a **more efficient alternative** to iptables, used for high-performance clusters.

How IPVS Works

- Instead of iptables, kube-proxy creates **IPVS rules** using the **Linux kernel's load balancing feature**.
- IPVS uses **Direct Server Return (DSR)**, which is **faster than iptables**.
- IPVS creates a **virtual IP** (VIP) and maps it to backend pods.

Example of IPVS Rule Setup

```
ipvsadm -A -t 10.96.0.10:80 -s rr
ipvsadm -a -t 10.96.0.10:80 -r 10.244.1.2:80 -m
ipvsadm -a -t 10.96.0.10:80 -r 10.244.2.3:80 -m
```

Explanation

1. 10.96.0.10:80 is mapped to backend pods.
2. Traffic is distributed **round-robin (-s rr)** to Pod-A and Pod-B.
3. **"-m" mode** ensures packets are sent to the backend pod via **Masquerading**.

Comparison of iptables vs. ipvs

Feature	iptables Mode 🔥	ipvs Mode 🚀

Performance	Slower, CPU-intensive	Faster, optimized for large clusters
Scalability	Struggles with large services	Can handle thousands of backends
Load Balancing	Random probability selection	Multiple strategies (RR, Least Connections, etc.)
Kernel Dependency	No special kernel features needed	Requires **IPVS kernel module**

💡 **When to use IPVS?**
- When you have a **large cluster** (1000+ services).
- When you need **high-performance load balancing**.
- When using **advanced scheduling algorithms**.

3. What Happens if a Node Doesn't Have the Target Pod?
- kube-proxy forwards the traffic **to another node** that has the correct pod.
- This happens **transparently** via **iptables/ipvs**, so the client doesn't know which pod it hit.
- If all backend pods are down, requests will fail with a **503 Service Unavailable**.

Want to Check Your Cluster's Mode?
Run the following command:

kubectl get configmap -n kube-system kube-proxy -o yaml | grep mode
If it returns mode: "ipvs", you're using **IPVS mode**; otherwise, it's **iptables**.

Q28. What is a ConfigMap in Kubernetes?

Why Use ConfigMaps?
- To **store environment variables** separately from application logic.
- To **change configuration without rebuilding images**.
- To **share common settings** across multiple pods.
- To **mount configuration files** inside containers.

Example 1: Creating a ConfigMap using YAML
Step 1: Define a ConfigMap
Create a file configmap.yaml:
yaml
CopyEdit
```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_NAME: "MyK8sApp"
```

```
  LOG_LEVEL: "debug"
  DATABASE_URL: "postgres://user:pass@db-service:5432/mydb"
```
Apply the ConfigMap:
```
sh
CopyEdit
kubectl apply -f configmap.yaml
```

Example 3: Mounting a ConfigMap as a File

You can also mount a ConfigMap as a file inside a pod.

Step 1: Create a ConfigMap

```
yaml
CopyEdit
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  config.json: |
    {
      "logLevel": "debug",
      "database": {
        "host": "db-service",
        "port": 5432
      }
    }
```

Step 2: Mount the ConfigMap in a Pod

```
yaml
CopyEdit
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 1
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app-container
          image: my-app-image
          volumeMounts:
            - name: config-volume
              mountPath: "/etc/config"
      volumes:
        - name: config-volume
          configMap:
            name: app-config
```

What happens?

- The file /etc/config/config.json will be created inside the container with the contents of the ConfigMap.

Key Differences Between ConfigMap and Secrets

Feature	ConfigMap	Secret

Data Type	Plain text	Base64 encoded
Use Case	Non-sensitive config	Sensitive data (passwords, API keys)
Mounted As	Env vars or files	Env vars or files
Security	Not encrypted	More secure, supports encryption

Q29. What is Job and CronJob in k8s ? How does it differ from from pod, deployment ,. service etc.

1️⃣ **What is a Job?**

A **Job** is a Kubernetes resource that **runs a pod to completion**. It ensures that a specific task is completed successfully a certain number of times. Once a Job finishes, the Pod does **not restart**.

Example: Running a Data Processing Job

Here's an example Job that runs a **Python script** inside a Pod:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: data-processing-job
spec:
  completions: 3  # Run this Job 3 times
  parallelism: 2  # Run 2 Jobs in parallel
  template:
    metadata:
      name: data-processing-pod
    spec:
      containers:
      - name: processor
        image: python:3.9
        command: ["python", "-c", "print('Processing data...')"]
      restartPolicy: Never  # Job should not restart on failure
```

Key Features of Jobs:

✅ Runs a task until it **completes successfully**

✅ Does **not** restart on failure (unless specified)

✅ Can run multiple **parallel** instances

2️⃣ **What is a CronJob?**

A **CronJob** is like a **Linux cron job**, used for scheduling Jobs to run at specific intervals.

Example: Running a Backup Every Day at Midnight

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: daily-backup
spec:
  schedule: "0 0 * * *"  # Run at midnight every day
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: busybox
            command: ["sh", "-c", "echo 'Running database backup'"]
          restartPolicy: OnFailure  # Restart on failure
```

Key Features of CronJobs:

✅ Runs Jobs at **specific times** (like a cron schedule)

✅ Can be **recurring** (daily, hourly, etc.)

✅ Automatically creates **Jobs** at scheduled times

3️⃣ Differences Between Pod, Deployment, Job, CronJob, and Service

Feature	Pod	Deployment	Job	CronJob	Service
Purpose	Run a single instance of an app	Maintain long-running apps	Run a task to completion	Run jobs on a schedule	Expose a service inside/outside cluster
Lifecycle	Short/long-lived	Long-lived	Short-lived	Recurring	Persistent
Restart Policy	Yes, if specified	Yes (managed by ReplicaSet)	No, unless specified	No, unless specified	N/A
Scaling	No	Yes (replicas)	No (unless parallelism is set)	No (creates Jobs instead)	N/A
Example	A single containerized app	A web app with multiple replicas	A script that runs once	A daily database backup	A load balancer for an app

4️⃣ When to Use Job vs. CronJob

Use Case	Use Job?	Use CronJob?
One-time data migration	✅ Yes	❌ No
Running a script every 5 minutes	❌ No	✅ Yes
Sending an email once	✅ Yes	❌ No
Performing a scheduled backup	❌ No	✅ Yes

Q30. What is serviceaccount.yml. What is Kubernetes SA for my service. What roles do it mention and who are the principals here. How does it differ from GCP Service account for project ?

ServiceAccount in Kubernetes vs. GCP Service Account

A **ServiceAccount** in Kubernetes (K8s) is used to **authenticate** a Pod within the cluster, while a **GCP Service Account** is used to authenticate a workload to Google Cloud services.

1️⃣ What is serviceaccount.yml in Kubernetes?

A **Kubernetes ServiceAccount (SA)** allows Pods to interact with the **Kubernetes API** securely. By default, every Pod gets assigned the **default ServiceAccount**, but you can create custom ServiceAccounts with specific permissions.

Example: serviceaccount.yml

yaml
CopyEdit

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-service-account
  namespace: default
```

- This **creates a ServiceAccount** named my-service-account in the default namespace.
- It does **not** define any permissions by itself.

2️⃣ Roles & RoleBindings for ServiceAccounts

A **ServiceAccount** needs **Role** or **ClusterRole** permissions to perform actions inside the cluster.

Example: Assigning Permissions to a ServiceAccount

yaml
CopyEdit

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

- This **Role** allows reading (get, list) Pods.

Binding the Role to a ServiceAccount

```
yaml
CopyEdit
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: my-service-account
  namespace: default
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

- This **RoleBinding** assigns the pod-reader role to the my-service-account ServiceAccount.

3 Who Are the Principals in Kubernetes SA?

In Kubernetes, **principals** are **ServiceAccounts or Users** that interact with the API.

- **Human Users**: Authenticate via kubectl (e.g., IAM users in cloud-based clusters).
- **ServiceAccounts**: Used by Pods to interact with Kubernetes API.
- **ClusterRoles**: Permissions that can apply **cluster-wide**.

4 Kubernetes Service Account vs. GCP Service Account

Feature	Kubernetes Service Account	GCP Service Account
Purpose	Authenticates Pods to access Kubernetes resources	Authenticates workloads to access GCP APIs
Scope	Inside a Kubernetes cluster	Across Google Cloud services
Permissions	Defined via **RBAC (Role/ ClusterRole)**	Defined via **IAM roles**
Identity	Identified within the cluster	Identified by **email & private key**
Use Case	Allow a Pod to query Kubernetes API (e.g., list pods)	Allow a Pod to access GCP services (e.g., Cloud Storage)
Example Role	Read-only access to Pods	roles/storage.objectViewer for GCS

5 How Does a Kubernetes Service Account Differ From a GCP Service Account?

1. **Kubernetes Service Account**
   - Used **inside** a Kubernetes cluster.
   - Pods authenticate to Kubernetes API using it.
   - Uses **RBAC (Role-Based Access Control)** for permissions.
1. **GCP Service Account**
   - Used **outside** Kubernetes (to access GCP services).
   - Used for authentication with **Google Cloud APIs**.
   - Uses **IAM policies** for access control.

6 When to Use Which?

Scenario	Use Kubernetes SA?	Use GCP SA?
Pod needs to list other Pods	✅ Yes	❌ No
Pod needs to access a Google Cloud Storage bucket	❌ No	✅ Yes
Pod wants to update a ConfigMap	✅ Yes	❌ No
Pod wants to send logs to Stackdriver	❌ No	✅ Yes

7 Attaching a GCP Service Account to a Kubernetes Pod

To allow a **Kubernetes Pod** to access **Google Cloud services**, you need to:

1. Create a **GCP Service Account**.
2. Bind it to a **Kubernetes Service Account**.
3. Use **Workload Identity** to authenticate.

yaml
CopyEdit

```
apiVersion: iam.cnrm.cloud.google.com/v1beta1
kind: IAMPolicyMember
metadata:
  name: workload-identity-binding
spec:
  member: "serviceAccount:my-k8s-sa@my-project.iam.gserviceaccount.com"
  role: "roles/storage.objectViewer"
  resourceRef:
    kind: IAMServiceAccount
    external: "projects/my-project/serviceAccounts/my-gcp-sa@my-
project.iam.gserviceaccount.com"
```

Final Thoughts

- **Use a Kubernetes Service Account** when a Pod needs to interact with **Kubernetes resources**.
- **Use a GCP Service Account** when a Pod needs to interact with **Google Cloud services** (e.g., GCS, BigQuery).
- **RBAC manages Kubernetes SA**, while **IAM manages GCP SA**.

Q31. What is statefulset in k8s ? How does it differ from from pod, deployment ,. service etc.

A **StatefulSet** in Kubernetes is a workload resource used to **manage stateful applications**. It provides each Pod with a **unique, stable identity and persistent storage** across reschedules, making it ideal for databases and distributed applications.

1 What is a StatefulSet?

A **StatefulSet** ensures that:

1. Each Pod gets a **stable network identity** (e.g., pod-0, pod-1).
2. Each Pod gets a **persistent storage volume** (even if rescheduled).

3. Pod creation and deletion occur **sequentially** (not parallel).

2️⃣ Key Differences: StatefulSet vs. Pod vs. Deployment vs. Service

Feature	Pod	Deployment	StatefulSet	Service
Purpose	Single instance of an application	Manages stateless replicas	Manages stateful applications	Exposes a set of Pods
Pod Identity	Ephemeral (changes on restart)	Random pod names	Stable pod names (pod-0, pod-1, etc.)	Provides a DNS name
Scaling	Not scalable itself	Scales easily, but replicas are interchangeable	Scales while maintaining identity	Load balances between pods
Storage	Lost when pod dies	Lost when pod dies	Each Pod gets a **Persistent Volume**	Does not manage storage
Ordering	No ordering	No ordering	**Ordered creation & deletion**	N/A
Use Case	Single instance apps	Stateless apps (e.g., Web APIs)	Stateful apps (e.g., Databases)	Exposes applications

3️⃣ Example: StatefulSet for a Database

This example deploys a **3-node Redis cluster** with stable Pod names (redis-0, redis-1, redis-2).

YAML: StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  serviceName: redis
  replicas: 3
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:latest
        ports:
        - containerPort: 6379
        volumeMounts:
        - name: redis-data
          mountPath: /data
  volumeClaimTemplates:
  - metadata:
```

```
    name: redis-data
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 1Gi
```

What Happens?

1. **Creates 3 Pods** (redis-0, redis-1, redis-2) sequentially.
2. Each Pod gets a **persistent volume (redis-data)**.
3. Even if a Pod dies, its **data is retained** in storage.

4 How StatefulSet Works Internally

A. Pod Naming & DNS

- A Pod in a StatefulSet is assigned a **stable DNS**:

  redis-0.redis.default.svc.cluster.local
- redis-1.redis.default.svc.cluster.local
-

  This means redis-0 can talk to redis-1 without worrying about IP changes.

B. Persistent Storage

- Each Pod gets a **Persistent Volume Claim (PVC)**.
- Even if redis-0 crashes, its **data remains intact**.

5 When to Use StatefulSet vs. Deployment?

Use Case	StatefulSet ✅	Deployment ✅
Databases (e.g., MySQL, MongoDB)	✅ Yes	❌ No
Message Queues (e.g., Kafka, RabbitMQ)	✅ Yes	❌ No
Leader-Follower Apps (e.g., Zookeeper)	✅ Yes	❌ No
Stateless Web Apps	❌ No	✅ Yes
Microservices	❌ No	✅ Yes

6 Does StatefulSet Need a Service?

Yes!

- It needs a **Headless Service** (serviceName: redis) to expose its Pods.
- A **headless service** (ClusterIP: None) allows Pods to be individually addressable.

Example: Headless Service

```
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
```

```
clusterIP: None
selector:
  app: redis
ports:
- port: 6379
```

7️⃣ StatefulSet Scaling & Rolling Updates

- Scaling is done **one Pod at a time**.
- Updates are done **sequentially** (pod-0 → pod-1 → pod-2).
- If redis-0 is down, redis-1 **won't start** until redis-0 is healthy.

8️⃣ Key Takeaways

✔ StatefulSet is used for databases & stateful applications.
✔ It guarantees stable Pod identity & persistent storage.
✔ Pods are created & deleted sequentially.
✔ Needs a headless service for stable DNS resolution.

———

📌 Summary

Question	Answer
Can multiple apps (Python, React, Go) run on a single node?	✅ Yes, if the node has enough resources.
What happens if replicaSet=2?	✅ Pods try to scale within the node first. If there's not enough space, some pods **stay pending** unless autoscaling is enabled.
How to check which pod is running on which node?	kubectl get pods -o wide
Where do control plane and data plane lie?	Control plane (API, scheduler) is managed by GKE; Data plane (nodes) runs the workloads.

✅ **Metrics Server runs as a Deployment inside the cluster.**
✅ **It collects live CPU/memory usage from nodes and pods.**
✅ **Used by HPA, VPA, and kubectl top to monitor cluster performance.**
✅ **Does NOT store historical data** (use Prometheus for that).

Level	What cgroups Track?	Example Path
Container Level	CPU & Memory limits for individual containers	/sys/fs/cgroup/cpu/docker/<container-id>/

Pod Level	CPU & Memory limits for all containers in a Pod	/sys/fs/cgroup/cpu/kubepods/pod<UID>/
Node Level	Tracks CPU & memory usage of all Pods on a Node	/sys/fs/cgroup/cpu/kubepods/

✅ **cgroups help Kubernetes enforce resource limits at Container, Pod, and Node levels.**

✅ **cgroups + Metrics Server + HPA = Pod autoscaling based on real-time usage.**

✅ Containers in **the same Pod** can share storage if they mount the same volume.

✅ Containers in **different Pods** cannot share storage (unless using networked storage).

✅ **emptyDir**, PersistentVolumeClaim, and hostPath enable storage sharing.

✅ **Pods do not share localhost with other Pods** (but do inside the same Pod).

✅ **Sidecar is deployed at the Pod level (not Node or Cluster).**

✅ **Runs alongside the main container in the same Pod.**

✅ **Enhances logging, monitoring, security, and networking.**

✅ **Common example: Istio service mesh injects an Envoy proxy as a sidecar.**

Question	Answer
Control Plane and Data Plane - Nodes or Pods?	✅ Nodes (Control Plane = Managed by GKE, Data Plane = Worker Nodes running Pods)
Deployment Scaling - Pod or Node Scaling?	✅ HPA scales Pods, ✅ Cluster Autoscaler scales Nodes
Does --enable-autoscaling mean node scaling?	✅ Yes, new nodes are added when needed
Are pods copied when nodes scale?	❌ No, new pods are scheduled on new nodes, existing pods remain unless rescheduled
How does traffic split across nodes?	✅ Load Balancer sends traffic to different pods across nodes
Where does Load Balancer exist?	✅ External (Google Load Balancer) or ✅ Internal (Service in Kubernetes)

Question	Answer
Q9: Is ClusterIP a Service? Inside or Outside the cluster?	✅ Yes, it is a Kubernetes Service that is strictly inside the cluster
Q10: How does KubeDNS work?	✅ It resolves service names to Pod IPs inside the cluster for stable communication
Q11: What happens when I create a GKE cluster?	✅ GKE provisions a managed Control Plane and allocates Worker Nodes

Q11b: Who manages resource allocation in GKE?	✅ GKE handles Control Plane setup, VM provisioning, networking

Question	Answer
Q12: What is Fleet? What is Namespace?	✅ Fleet manages multiple GKE clusters; ✅ Namespace organizes workloads inside a single cluster.
Q13: What happens when I specify no-proxy in values.yml?	✅ It tells Kubernetes which destinations should bypass the proxy. ✅ kube-proxy or an external proxy handles it.
Q14: Who maintains the routing table? Is there a router inside Kubernetes?	✅ kube-proxy manages routing via iptables/ipvs. ✅ CNI plugins handle Pod networking. ✅ VPC routing is used in cloud setups.

Component	Responsibility
Cloud LoadBalancer	Manages external traffic and forwards it to Nodes
kube-proxy	Handles service routing, NAT, and IP forwarding
KubeDNS/CoreDNS	Resolves Service names to ClusterIP addresses
Kubernetes Scheduler	Decides where new Pods should run
Kubernetes API Server	Manages all cluster resources